ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

SOFTWARE REQUIREMENT SPECIFICATION

CAMPUSHIRE

COLLEGE PLACEMENT AND INTERNSHIP MANAGEMENT SYSTEM

Prepared in IEEE Std 830-1998 style

Under the Guidance of

Er. Ram Babu Buri

Assistant Professor

Submitted By

Team Member	RTU Roll No.
Virendra Singh Chundawat	24EARAD179
Vaibhav Charan	24EARAD170
Yash Kumar	24EARAD182
Udit Bharadwaj	24EARAD167
Rahul Tailor	24EARAD130

Document Status and Interpretation

This Software Requirements Specification (SRS) document has been prepared for **CampusHire — College Placement and Internship Management System**, based on the project report, the research paper, the system architecture, the implemented features and the team's recorded measurements. It follows the recommended structure of IEEE Std 830-1998 and provides a formal description of the software requirements, interfaces, database design, operating environment and system constraints.

The primary purpose of this document is to establish a clear understanding of the system's expected functionality among developers, testers, faculty evaluators and future maintainers. It serves as a reference for software development, testing, verification and project evaluation, ensuring consistency between the proposed requirements and the implemented application.

The document includes technical diagrams such as the System Context Diagram, Use Case Diagram, Data Flow Diagrams (DFDs), Flowcharts, Sequence Diagrams, UI screenshots, Deployment Topology, the Application Lifecycle (state) diagram and the Entity–Relationship Diagram (ERD). These diagrams illustrate the system architecture, user interactions, data processing workflows, communication between components and database relationships.

CampusHire is an academic prototype that supports the campus recruitment lifecycle for three self-service roles — student, company recruiter and Training and Placement Officer (TPO) — with server-side role enforcement, parameterized data access, server-side eligibility checking and a five-state application lifecycle. It is deployed as a student prototype and has not been used for a real placement season. Measurements quoted in this document were recorded by the team in single environments; they are not a load test, a user study or a security certification. Items that could not be confirmed from the available project material are marked **[CONFIRM]** and must be verified against the source code before final submission.

TABLE OF CONTENTS

Section	Particulars	Page No.
1.	**INTRODUCTION**	**1**
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms, and Abbreviations	2
1.4	Overview	2
2.	**OVERALL DESCRIPTION**	**3**
2.1	Product Perspective	3
2.2	Product Features	3
2.3	User Classes and Characteristics	4
2.4	Operating Environment	5
2.5	Design and Implementation Constraints	5
2.6	Assumptions and Dependencies	5
3.	**SPECIFIC REQUIREMENTS**	**7**
3.1	External Interfaces	7
3.2	Functional Requirements	9
3.3	Performance Requirements	16
3.4	Logical Database Requirements	16
3.5	Software System Attributes	18
4.	**APPENDICES & REFERENCES**	**20**
A	Key Business Rules	20
B	Edge Cases and Expected Behaviour	20
C	Recorded Verification Cases	21
D	Requirement Traceability Overview	21
E	Deployment and Operational Notes	21
5.	**REFERENCES**	**23**

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, interface, data, performance, security, reliability, maintainability and availability requirements for CampusHire, a Java web application for role-separated campus recruitment and placement lifecycle management. It is intended for the project team, faculty evaluators, developers, testers, database designers and future maintainers.

The specification provides a common baseline for implementation, verification and acceptance. It covers the public landing page, student registration and profile, company registration and TPO approval, job and internship postings, server-side eligibility filtering, application submission, recruiter review with status updates, TPO administration and the access-control pipeline that separates the three roles.

1.2 Scope

CampusHire gives students, company recruiters and the placement office a single application in place of spreadsheets, e-mail circulars and notice boards. It lets students see only the postings they are eligible for, enforces one application per student per posting, lets recruiters review applicants and record decisions, and lets students see the current status and remarks of every application. The TPO verifies companies, manages users and views placement statistics.

Included in Scope	Explicitly Out of Scope for the Current Version
User registration and login for three roles (student, company, admin/TPO) with session-based role access.	A production-grade identity provider, SSO, MFA or enterprise directory integration.
Company registration that remains pending until the TPO approves it.	Independent verification of every employer or of student-entered CGPA by the software itself.
Student profile (roll number, department, CGPA, passing year, résumé link).	Résumé file storage; only a résumé link is stored.
Job and internship postings with cut-off criteria and a deadline.	Measured placement improvement or hiring-outcome guarantees.
Server-side eligibility filtering and duplicate-application prevention.	Automated e-mail or SMS notification, AI matching or recommendation features.
Five-state application lifecycle visible to all three actors.	Legal-compliance certification, security certification or independent audit.
Servlet filter (AuthFilter) role enforcement and view templates under WEB-INF.	High availability, clustering, automated disaster recovery or an uptime SLA.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
CampusHire	College placement and internship management system described by this SRS.
ACID	Atomicity, Consistency, Isolation, Durability — transaction guarantees of the relational database.
AuthFilter	Servlet filter that compares the session role with the requested path prefix before any servlet runs.
CGPA	Cumulative Grade Point Average (0.00–10.00 in the registration form).
DAO	Data Access Object; Java class that encapsulates database operations through JDBC.
Eligibility	The institution's or company's rule for who may apply to a given posting.
FSM	Finite State Machine; here, the application lifecycle.
JDBC	Java Database Connectivity.
JSP	JavaServer Pages; server-side view templates.
LPA	Lakhs per annum; unit used for job packages.
MVC	Model–View–Controller architectural pattern.
OWASP	Open Worldwide Application Security Project.
RBAC	Role-Based Access Control.
TiDB Cloud	MySQL-compatible serverless cloud database used in the hosted deployment.
TPO	Training and Placement Officer; the administrator role.
WAR	Web Application Archive; the deployable package built by Maven.

1.4 Overview

Section 2 describes the product context, capabilities, user classes, operating environment, constraints and assumptions. Section 3 specifies external interfaces, functional requirements with unique identifiers, performance expectations, database requirements and software quality attributes. Section 4 contains business rules, edge cases, recorded verification cases, traceability, deployment notes and references.

2. Overall Description

2.1 Product Perspective

CampusHire is a browser-accessible web application built on a three-tier MVC structure: JSP views and static assets (presentation), Jakarta Servlets 6.0 with an AuthFilter (business logic) and DAO classes with pure JDBC over MySQL 8.0 or TiDB Cloud Serverless (persistence). Maven packages the application into a single 2.45 MB WAR, which a multi-stage Docker build runs on Apache Tomcat 10.1. The deployed prototype is hosted on Render with a TiDB Cloud Serverless database; the same code runs against a local MySQL instance, selected by configuration in the DBConnection utility.

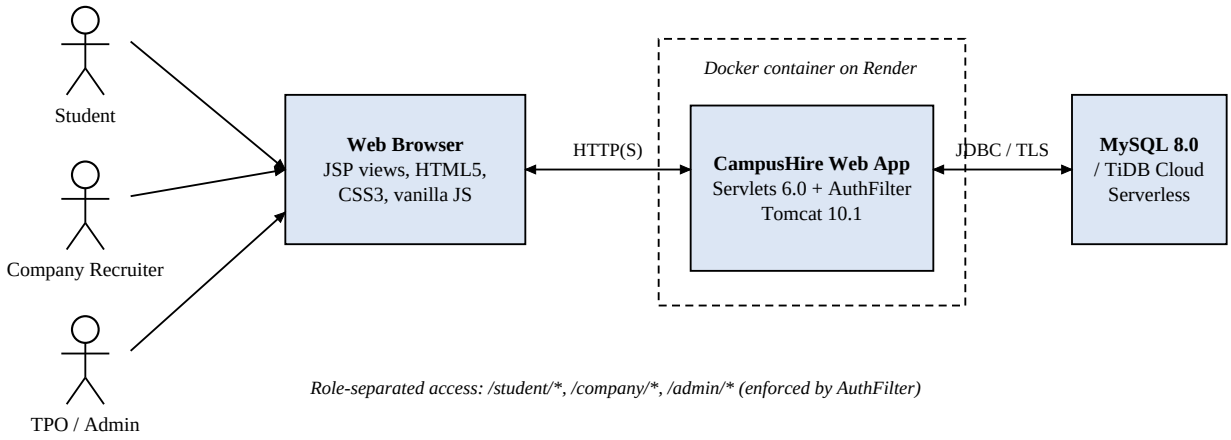


Figure 2.1: System Context Diagram

External dependencies are the user's browser, the servlet container, the configured database and, for the hosted deployment, the Render and TiDB Cloud services. No sensor hardware, e-mail gateway or third-party identity provider is used. The system deliberately uses no application framework such as Spring or Hibernate; it depends only on the Jakarta Servlet API, a servlet container, a JDBC driver and the Java standard library.

2.2 Product Features

- Public landing page "College Placement & Internship Management System" with counters, latest placement drives, featured internships and a "Why CampusHire?" panel.
- Student and company registration; role-tabbed login (Student, Company HR, Admin / TPO); student login by username, e-mail or roll number.
- Company approval workflow: a registered company cannot log in until the TPO marks it APPROVED.
- Student profile with roll number, department, CGPA, passing year and résumé link.
- Job postings (package in LPA, minimum CGPA, allowed branches, openings, deadline) and internship postings (monthly stipend, duration, minimum CGPA, deadline).
- Server-side eligibility filtering: Student.CGPA ≥ minimum CGPA and an allowed branch.
- One application per student per posting, enforced by a database uniqueness constraint.

- Five-state application lifecycle (PENDING, SHORTLISTED, INTERVIEW_SCHEDULED, ACCEPTED, REJECTED) with recruiter remarks visible to students.
- TPO administration of users and company approvals, and placement statistics.
- AuthFilter role enforcement and WEB-INF view protection against direct URL access.

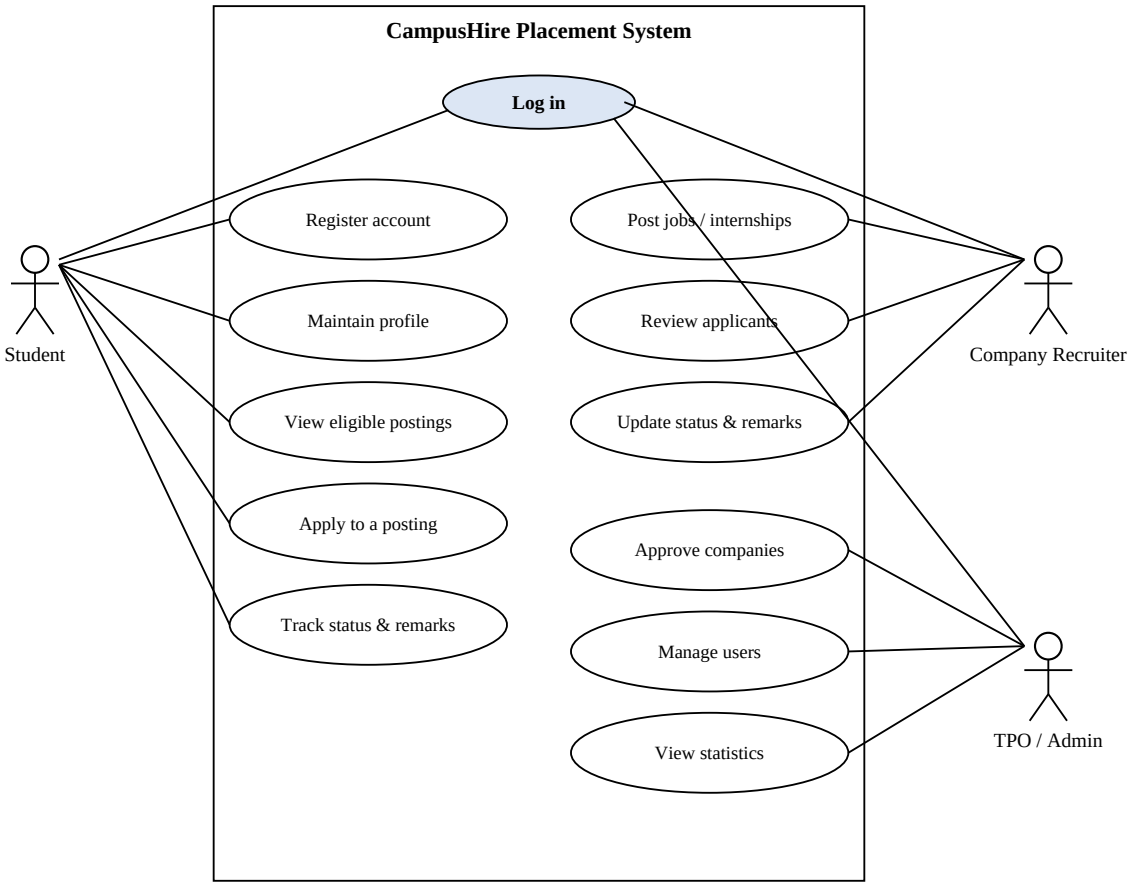


Figure 2.2: Use Case Diagram

2.3 User Classes and Characteristics

User Class	Typical Goals	Permissions / Characteristics
Student	Find postings they qualify for, apply, and follow application status and recruiter remarks.	Own profile and applications only; sees only eligible postings; cannot access /company/* or /admin/*.
Company Recruiter	Publish jobs and internships, review applicants and record hiring decisions.	Access only after TPO approval; sees only applicants to own postings; cannot access /student/* or /admin/*.
TPO / Administrator	Verify companies, manage users, enforce institutional rules and report outcomes.	Administrative access to /admin/*; views all students, companies, postings, applications and statistics.
Maintainer / Developer	Build, deploy, configure and extend the system.	Operational access to source, SQL scripts, Docker configuration and database; not an application role.

2.4 Operating Environment

Component	Requirement / Environment
Server runtime	Java with Jakarta Servlets 6.0 on Apache Tomcat 10.1 [CONFIRM: exact JDK version from pom.xml].
Build	Maven; produces one WAR (2.45 MB recorded).
Packaging / hosting	Multi-stage Docker image run on Render; local run with Tomcat 10.1.
Database	MySQL 8.0 (local) or TiDB Cloud Serverless over TLS (hosted); database campus_hire_db with six tables.
Client	Modern desktop or mobile browser with HTML5, CSS3 and JavaScript support; layouts must work on small screens.
Network	HTTP/HTTPS between browser and application; TLS between application and the cloud database.
Hardware	Any general-purpose computer or container host able to run Tomcat 10.1; recorded idle JVM memory is 168 MB local and 210 MB cloud.

2.5 Design and Implementation Constraints

- The backend uses Core Java, Jakarta Servlets 6.0, JSP and pure JDBC; no Spring, Hibernate or other application framework is used.
- All database access shall use prepared statements; concatenating user input into SQL is prohibited.
- View templates shall be placed under /WEB-INF/views/ so that the container does not serve them directly.
- Every protected path prefix (/student/*, /company/*, /admin/*) shall be covered by AuthFilter; a new route that is not added to the filter is unprotected.
- The current prototype stores a plain, unsalted SHA-256 digest in password_hash (CHAR(64)). This is a known weakness; production use shall replace it with a salted, deliberately slow algorithm such as bcrypt, scrypt, Argon2id or PBKDF2.
- Features that frameworks normally provide — CSRF protection, secure password encoders and input binding — must be implemented and tested by hand; CSRF, secure cookie flags and output-encoding coverage are to be verified [CONFIRM].
- Application status transitions are specified as one forward path; the code may enforce them only through the user interface [CONFIRM whether a server-side transition table exists].
- CGPA is entered by the student; eligibility is only as accurate as the entered value, so TPO verification against institutional records is a required procedure.

2.6 Assumptions and Dependencies

- The TPO approves companies promptly and verifies self-entered student CGPA against institutional records.
- Recruiters actually update application statuses; software cannot require this.
- The container, database and hosting service are available; free-tier hosting may incur cold starts and sleep.
- The deployed instance currently shows sample data in its counters and cards [CONFIRM]; it has not been used for a real placement season [CONFIRM].

- The student registration form collects mobile number, technical skills and address, which are not in the schema summary of Section 3.4 [CONFIRM: update schema].
- Performance targets in Section 3.3 are proposed acceptance targets; the recorded values are single-environment observations and not guarantees.
- Any use with real student data requires review under applicable data-protection law and institutional rules; no legal-compliance claim is made.

3. Specific Requirements

3.1 External Interfaces

3.1.1 User Interface

The UI shall provide a public landing page, a registration chooser, separate student and company registration forms, a role-tabbed login page and role-specific dashboards. Forms shall indicate required fields, show validation errors and remain usable on small screens. Figure 3.1 shows the implemented screens of the deployed prototype.

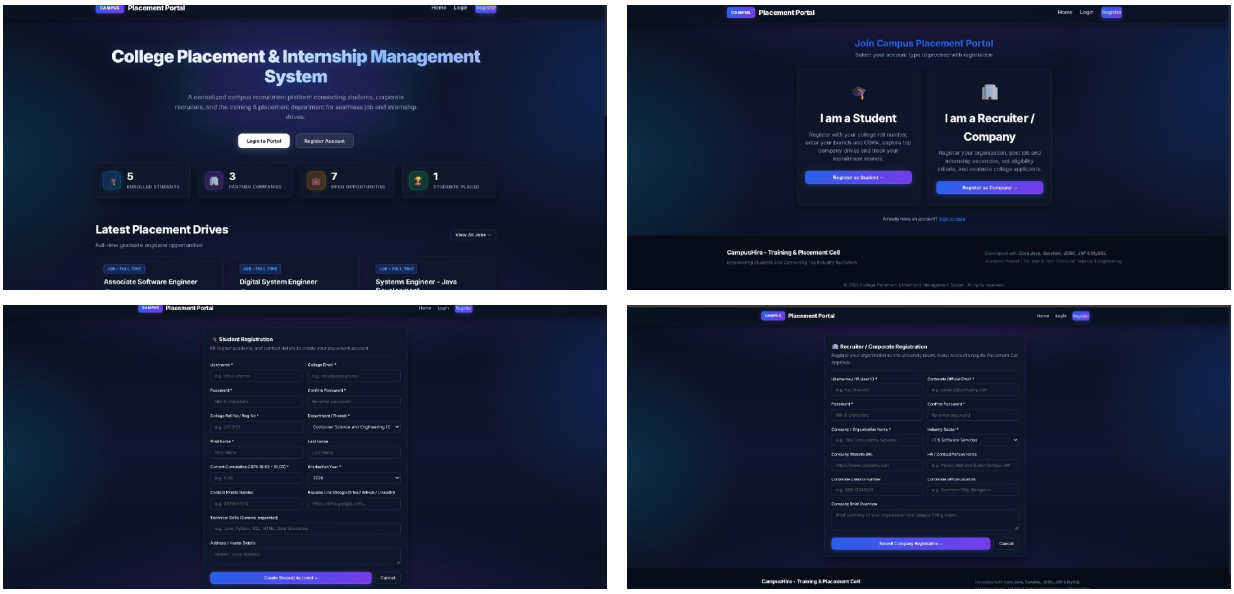


Figure 3.1: Implemented UI — landing page, registration chooser, student registration and company registration

UI Screen / Component	Required Behaviour
Landing page	Show title, counters (enrolled students, partner companies, open opportunities, students placed), latest placement drives with package, minimum CGPA and deadline, featured internships with monthly stipend, and entry points to login and registration.
Registration chooser	Offer two paths: “I am a Student” and “I am a Recruiter / Company”.
Student registration	Collect username, college e-mail, password and confirmation, roll number, department, first and last name, CGPA (0.00–10.00), graduation year, mobile number, résumé link, technical skills and address; mark required fields.
Company registration	Collect username / HR user ID, official e-mail, password and confirmation, company name, industry sector, website, HR contact person, contact number, office location and overview; state that accounts require Placement Cell approval.
Login	Provide three tabs — Student (username, e-mail or roll number), Company HR and Admin / TPO — each with username and password fields.
Student, recruiter and TPO dashboards	Show only the data and actions permitted to the role (Table 3.2). Detailed screens are not covered by the supplied screenshots [CONFIRM].

3.1.2 Hardware Interfaces

No direct hardware interface is required. The system runs on a standard server or container host and is accessed through a standard web browser.

3.1.3 Software Interfaces

Interface	Specification
Browser → application	HTTP(S) requests to servlets and JSP views; session cookie identifies the authenticated actor.
Application → database	Pure JDBC with prepared statements through DAO classes; MySQL 8.0 locally or TiDB Cloud Serverless over TLS.
AuthFilter → servlets	Filter intercepts every request, compares the session role with the path prefix and redirects unauthorised requests to login.
Build / runtime	Maven (WAR), Docker multi-stage image, Apache Tomcat 10.1.
Configuration	DBConnection selects local or cloud database from configuration, so the same code runs in both settings.

3.1.4 Communication Links and Network Topology

The browser communicates with the Tomcat application over HTTP (locally) or HTTPS (hosted on Render). The application reaches the database through JDBC; the hosted deployment uses TLS to TiDB Cloud. Database ports shall not be exposed to public clients, and production deployment shall use HTTPS throughout.

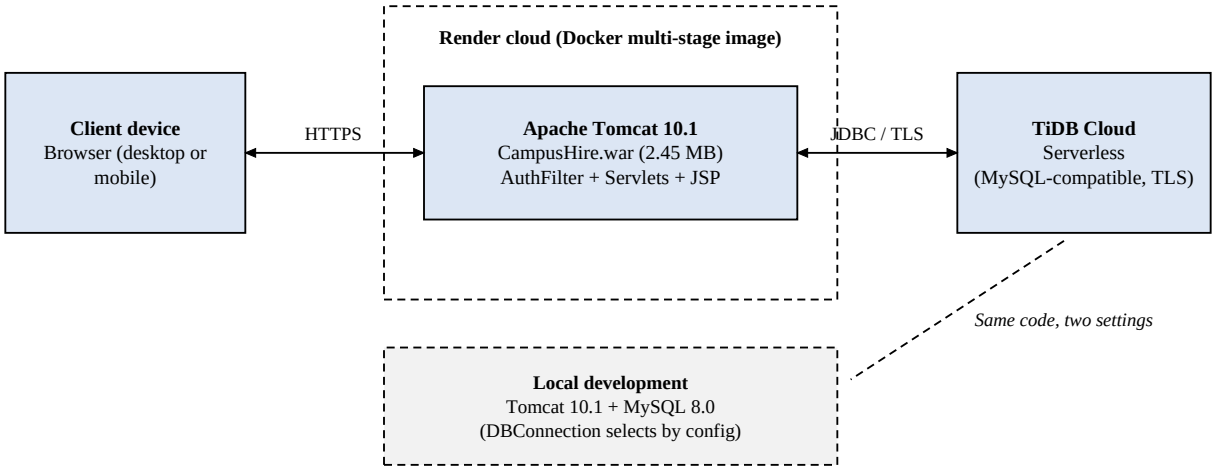


Figure 3.2: Deployment and Network Topology

3.1.5 Request Path Interface

CampusHire is servlet-based rather than a REST API. Access is organised by path prefix and enforced by AuthFilter. Exact servlet mappings and parameter names are not in the supplied material and must be confirmed from web.xml / annotations [CONFIRM].

Path / Component	Allowed Role	Purpose / Expected Result
/index.jsp (landing page)	Public	Show counters, latest drives, featured internships and entry points.

Path / Component	Allowed Role	Purpose / Expected Result
/login.jsp and AuthServlet	Public	Authenticate by role tab; create session with user id and role; reject invalid credentials, inactive accounts and unapproved companies.
Registration pages and AuthServlet	Public	Register a student or a company; company stays PENDING until approved.
/student/* (StudentServlet)	STUDENT	Profile, eligible postings, application submission, own application status.
/company/* (CompanyServlet)	COMPANY (approved)	Create postings, review applicants of own postings, update status with remarks.
/admin/* (AdminServlet)	ADMIN	Approve or reject companies, manage users, view statistics.
/WEB-INF/views/*	None (internal)	JSP templates; not served directly by the container.

3.2 Functional Requirements

The following functional requirements define the expected behaviour of CampusHire. Each requirement has a unique identifier to support traceability, implementation and testing.

3.2.1 Authentication and Authorization

FR-AUTH-01 — User Registration: The system shall allow a student or a company to register. Student registration shall require username, college e-mail, password with confirmation, roll number, department, first name, CGPA (0.00–10.00) and graduation year; company registration shall require username, official e-mail, password with confirmation, company name and industry sector. Duplicate usernames, e-mail addresses and roll numbers shall be rejected.

FR-AUTH-02 — User Login: The system shall authenticate users through a role-specific login (Student, Company HR, Admin / TPO) using username, e-mail or roll number (students) and password. The submitted password shall be hashed and compared with the stored digest; mismatches shall be rejected.

FR-AUTH-03 — Account Status: The system shall reject login for non-active accounts and for companies whose approval status is not APPROVED.

FR-AUTH-04 — Session Management: On successful login the system shall store the user id and role in the HTTP session and shall end the session on logout.

FR-AUTH-05 — Role-Based Access Control: AuthFilter shall intercept every request, read the role stored in the session and compare it with the requested path prefix (/student/*, /company/*, /admin/*); unauthorised requests shall be redirected to the login page.

FR-AUTH-06 — Direct URL Protection: The system shall block direct URL access to pages the current role may not view, including JSP templates, which shall reside under /WEB-INF/views/.

3.2.2 Student Profile and Postings

FR-STU-01 — Profile Maintenance: The system shall let students maintain a profile containing roll number, department, CGPA, passing year and résumé link.

FR-STU-02 — Eligible Postings: The system shall show a student only the job and internship postings for which the student is eligible (FR-ELG-01).

FR-STU-03 — Application Status View: The system shall show each student the current status and recruiter remarks of every application of that student.

FR-CMP-01 — Company Approval Gate: A newly registered company shall remain PENDING until the TPO approves it; it shall be unable to post until APPROVED.

FR-CMP-02 — Job Postings: Approved companies shall be able to create job postings with title, package (LPA), minimum CGPA, allowed branches, number of openings, deadline and status.

FR-CMP-03 — Internship Postings: Approved companies shall be able to create internship postings with title, stipend per month, duration in months, minimum CGPA, deadline and status.

FR-CMP-04 — Applicant Review: A recruiter shall see applicants only for postings owned by that recruiter's company.

FR-CMP-05 — Status Update: A recruiter shall update an applicant's status with remarks; the requested transition shall be checked against the lifecycle (Table 3.1) before it is stored.

3.2.3 Eligibility and Applications

FR-ELG-01 — Eligibility Predicate: A student shall be eligible for a job when Student.CGPA ≥ Job.min_cgpa and (Job.allowed_branches = "All Branches" or the student's department belongs to Job.allowed_branches). The same minimum-CGPA rule applies to internships.

FR-ELG-02 — Server-Side Enforcement: The eligibility check shall run on the server when postings are listed and again when an application is submitted, so that a client-side bypass cannot create an ineligible application.

FR-APP-01 — Application Submission: The system shall accept an application only from an authenticated student, for a posting that is open and before its deadline, and shall insert (student_id, posting_type, posting_id) with status PENDING using a prepared statement.

FR-APP-02 — Duplicate Prevention: The system shall enforce one application per student per posting through a uniqueness constraint that holds even under concurrent requests.

FR-APP-03 — Failure Handling: Failed validations or storage errors shall return an error without side effects and shall not display a success confirmation.

3.2.4 Application Lifecycle

FR-LIF-01 — Lifecycle States: Each application shall be in exactly one of five states: PENDING, SHORTLISTED, INTERVIEW_SCHEDULED, ACCEPTED or REJECTED.

FR-LIF-02 — Permitted Transitions: Only the transitions in Table 3.1 shall be accepted. [CONFIRM: whether the code enforces exactly these transitions or only sets the status field; if the latter, transitions are enforced by the user interface and not by a server-side transition table.]

FR-LIF-03 — Visibility: Students shall see the new status and remarks on the next page load; recruiters shall see statuses of their own applicants; the TPO shall see all.

Table 3.1: Application lifecycle

From	To	Actor
(new)	PENDING	Student
PENDING	SHORTLISTED	Recruiter
SHORTLISTED	INTERVIEW_SCHEDULED	Recruiter
INTERVIEW_SCHEDULED	ACCEPTED	Recruiter
INTERVIEW_SCHEDULED	REJECTED	Recruiter

3.2.5 Administration and Reporting

FR-ADM-01 — Company Verification: The TPO shall approve or reject company registrations (PENDING, APPROVED, REJECTED).

FR-ADM-02 — User Management: The TPO shall view and manage student and company users.

FR-ADM-03 — Placement Statistics: The TPO shall view placement statistics. Reports shall state their unit of analysis (a count of applications is not a count of placed students) [CONFIRM: reports actually implemented].

FR-UI-01 — Landing Page: The public landing page shall present counters, latest placement drives and featured internships, and links to login and registration.

FR-ENV-01 — Environment Abstraction: The system shall select a local MySQL database or a cloud TLS database from configuration without code changes.

3.2.6 Process Diagrams

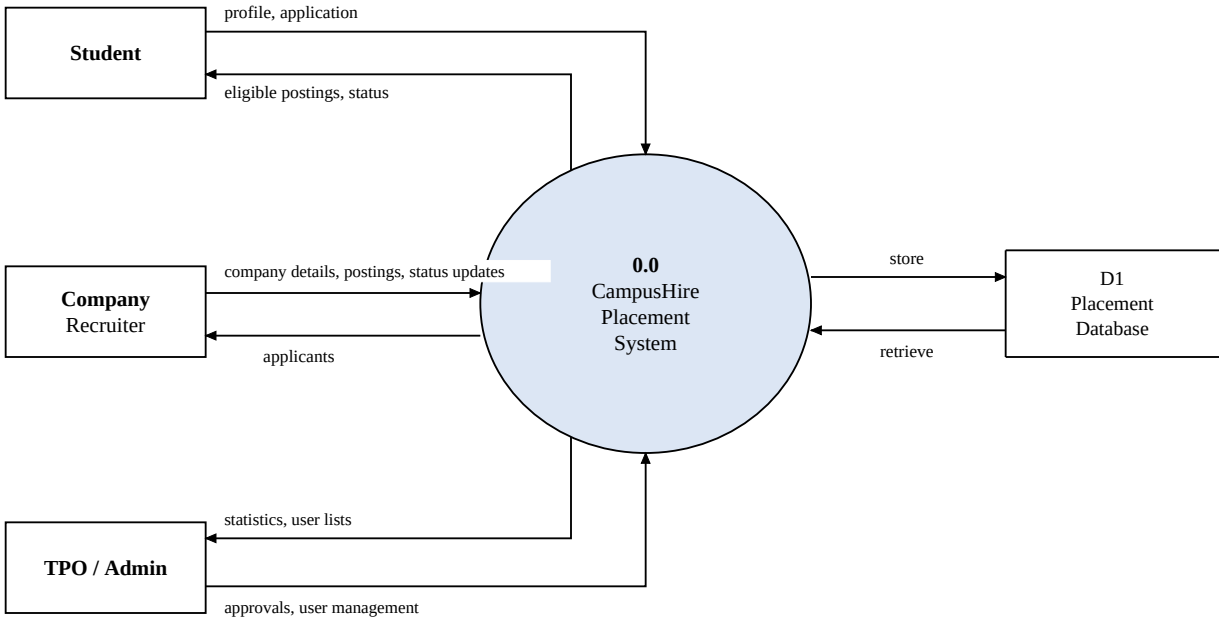


Figure 3.3: Data Flow Diagram — Level 0 (Context)

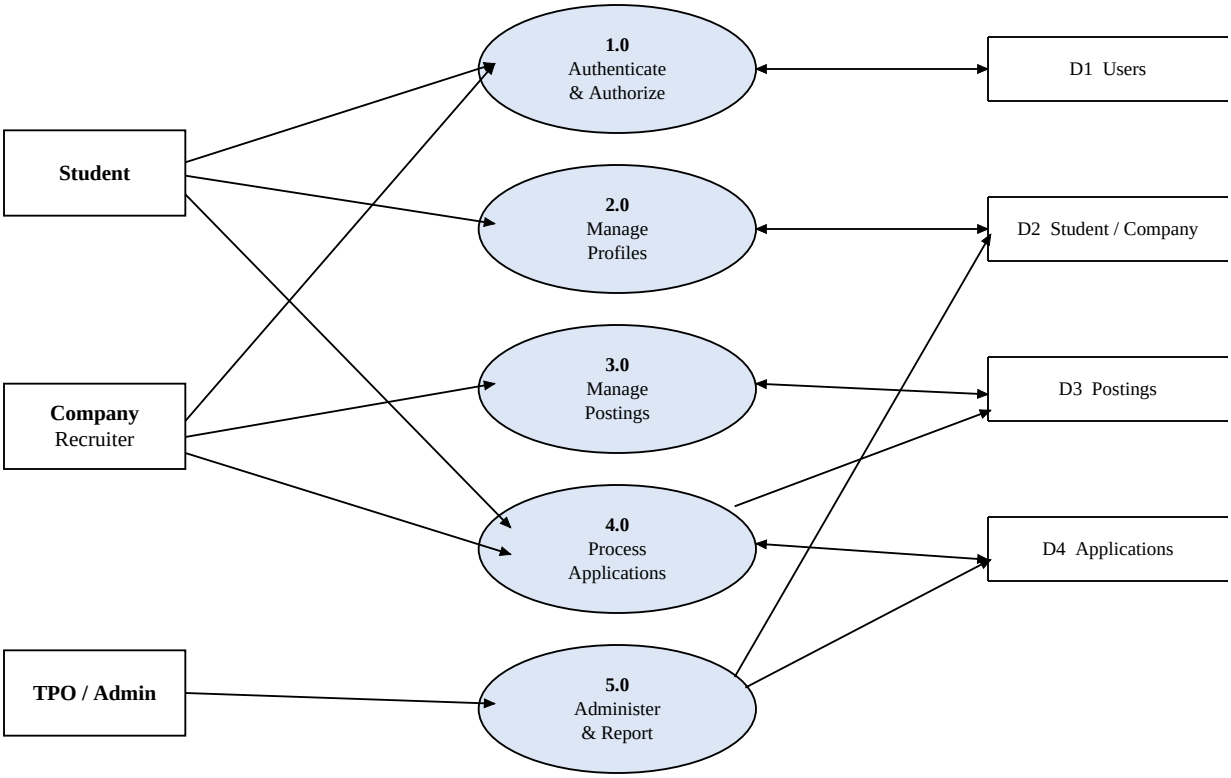


Figure 3.4: Data Flow Diagram — Level 1 (Major Processes)

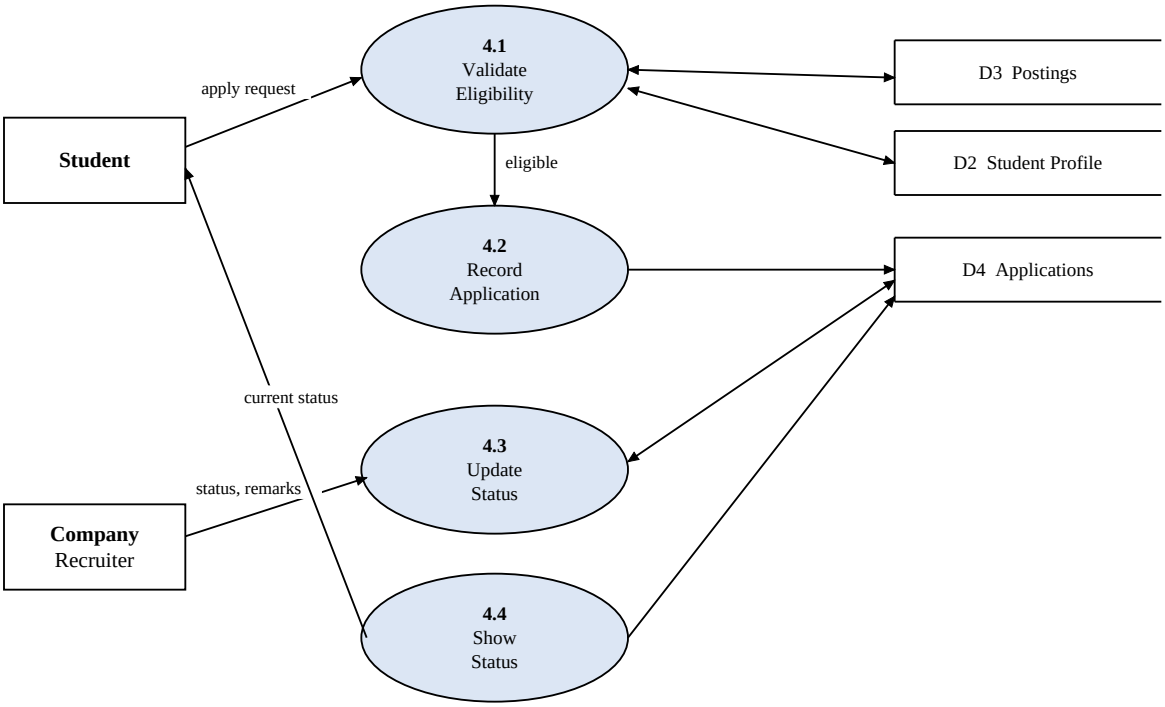


Figure 3.5: Data Flow Diagram — Level 2 (Process 4.0 Process Applications)

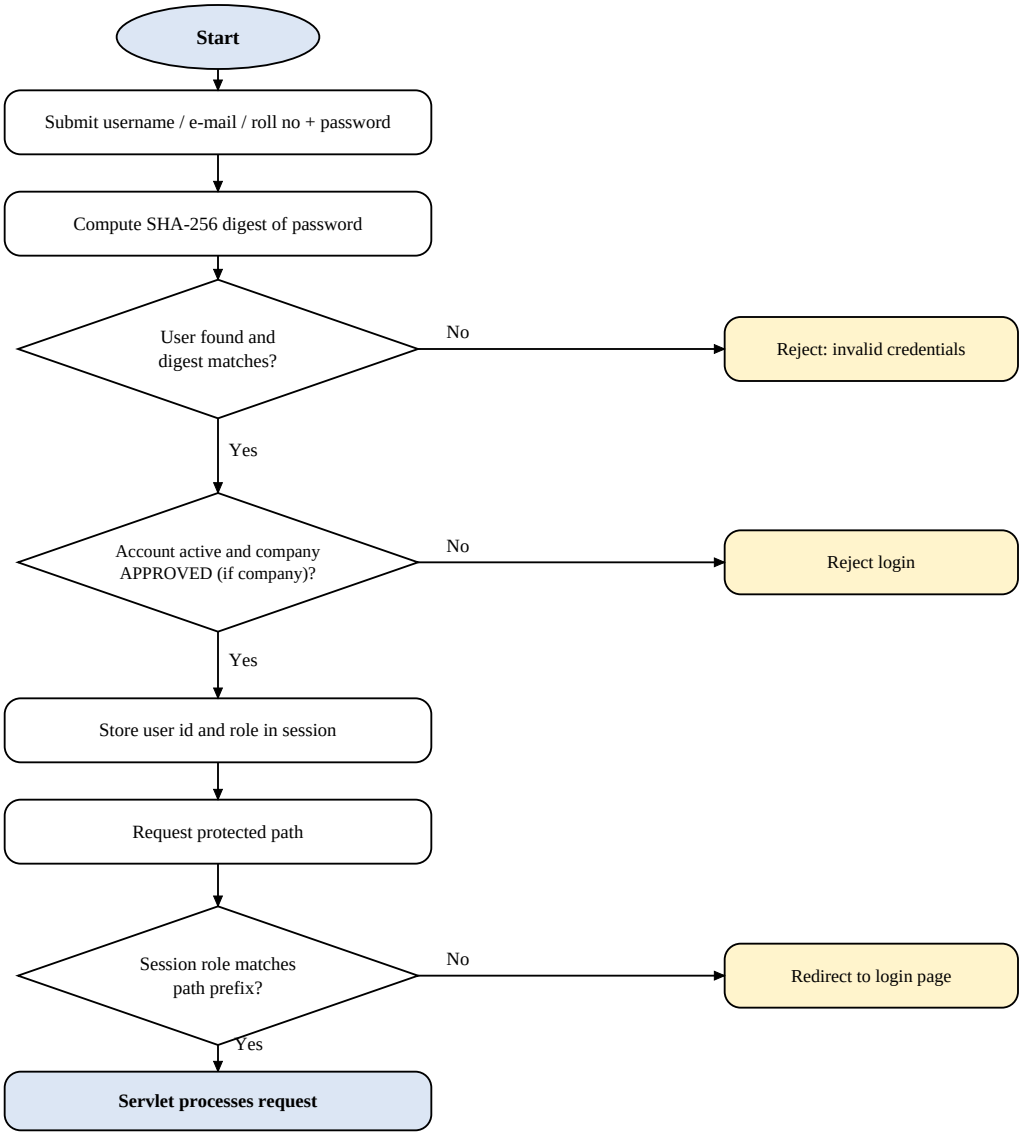


Figure 3.6: Flowchart — Login and Authorization

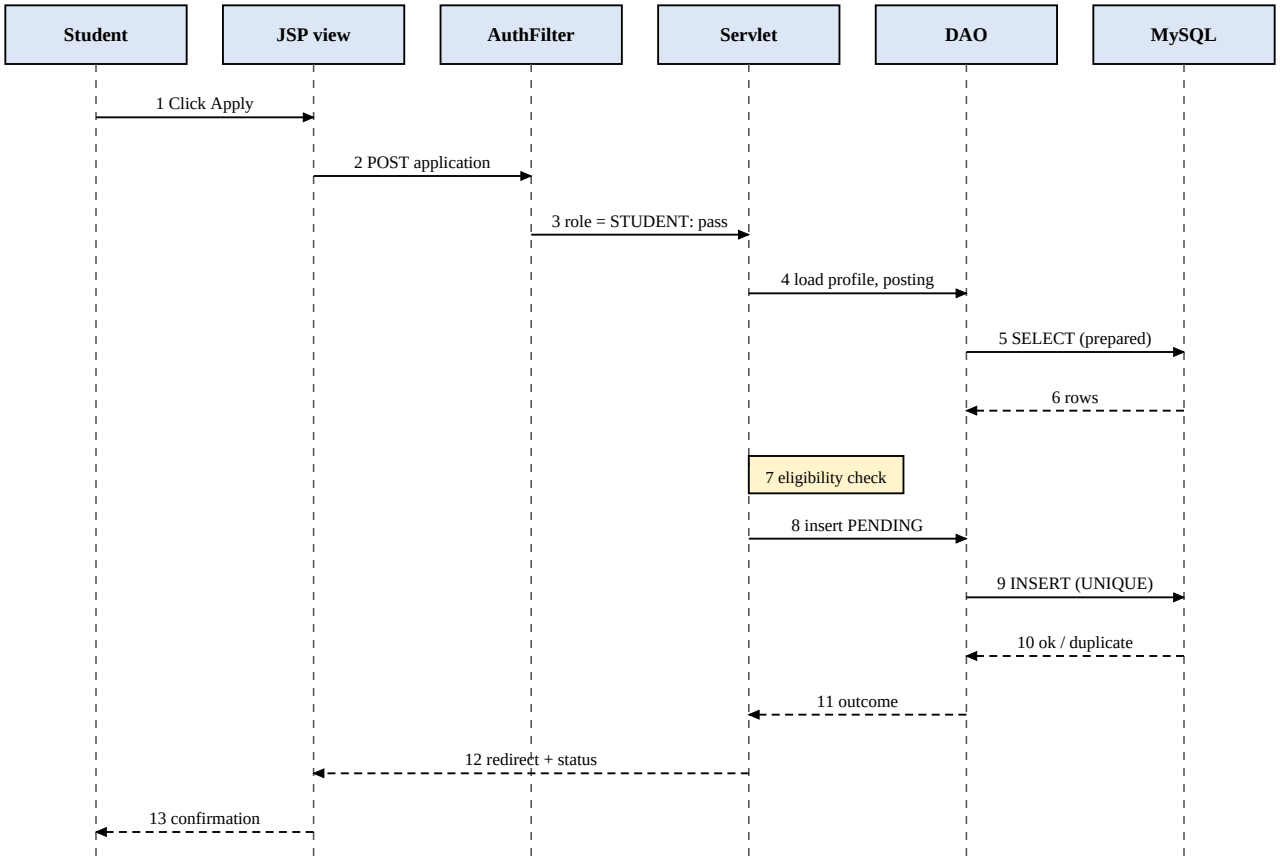


Figure 3.7: Sequence Diagram — Application Submission

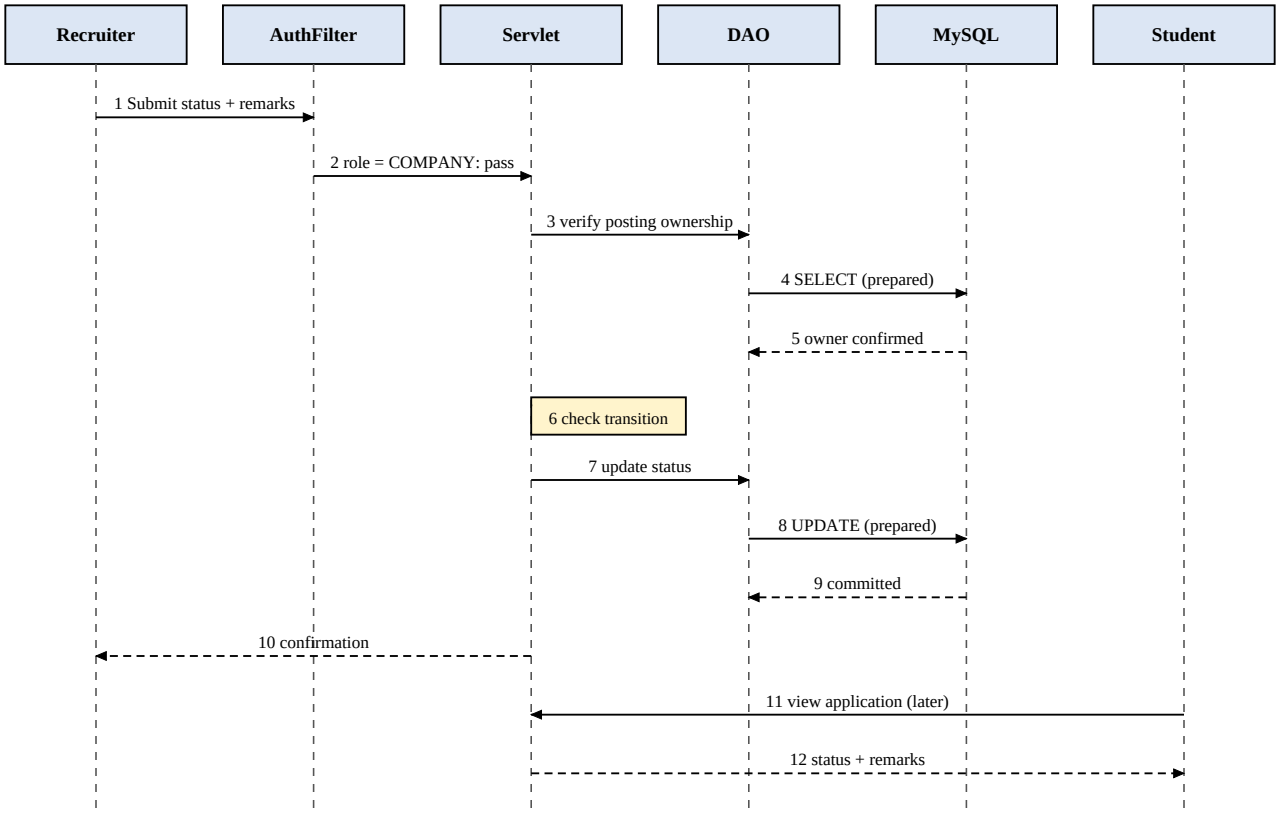
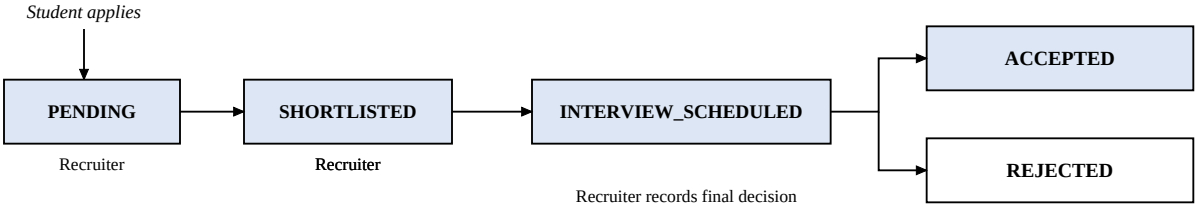


Figure 3.8: Sequence Diagram — Candidate Status Update



One forward path; ACCEPTED or REJECTED is final. Remarks are stored with each update.

Figure 3.9: State Diagram — Application Lifecycle

3.3 Performance Requirements

The project has no formal load test, concurrency test or framework baseline. The targets below are proposed acceptance targets for a student demonstration deployment; the “recorded value” column lists the team’s single-environment observations (local development machine and Render + TiDB Cloud) and is not a guarantee. Each target must be tested in the actual environment before being treated as a measured result.

ID	Metric	Proposed Acceptance Target	Recorded Value (Local / Cloud)
NFR-PERF-01	Page response, GET /	95% of ordinary read requests complete within 2 seconds under demo load.	14.2 ms / 185 ms (average)
NFR-PERF-02	Login request latency	Login completes within 2 seconds, including hashing and database round trip.	22.6 ms / 210 ms
NFR-PERF-03	Database query time	A single indexed query completes within 100 ms on the hosted database.	4.8 ms / 42.5 ms
NFR-PERF-04	Package size	Deployable WAR stays small enough for a free-tier container.	2.45 MB
NFR-PERF-05	Container startup	Application is ready within 30 seconds after container start.	1.75 s / 5.2 s
NFR-PERF-06	Idle memory	Idle JVM memory stays within the memory of a small container.	168 MB / 210 MB
NFR-PERF-07	Concurrent users	Support 25 concurrent sessions without failed requests (not yet tested).	Not measured
NFR-PERF-08	Error handling	Invalid input and unauthorised requests return a controlled response, not an unhandled server failure.	Direct-URL and injection tests: see Appendix C

The recorded values show that the prototype is small and responsive in the tested conditions. They do not show that it scales to many concurrent users, that it is free of vulnerabilities, or that it is smaller or faster than a framework-based alternative. Uptime was not measured over a defined period and no uptime figure is claimed.

3.4 Logical Database Requirements

The logical schema (database campus_hire_db) contains six tables with foreign keys and cascading deletes. Applications reference a posting by (posting_type, posting_id) and therefore have no foreign key to jobs or internships. The schema is described as normalized to Third Normal Form; this is the team’s design claim and has not been formally verified [CONFIRM].

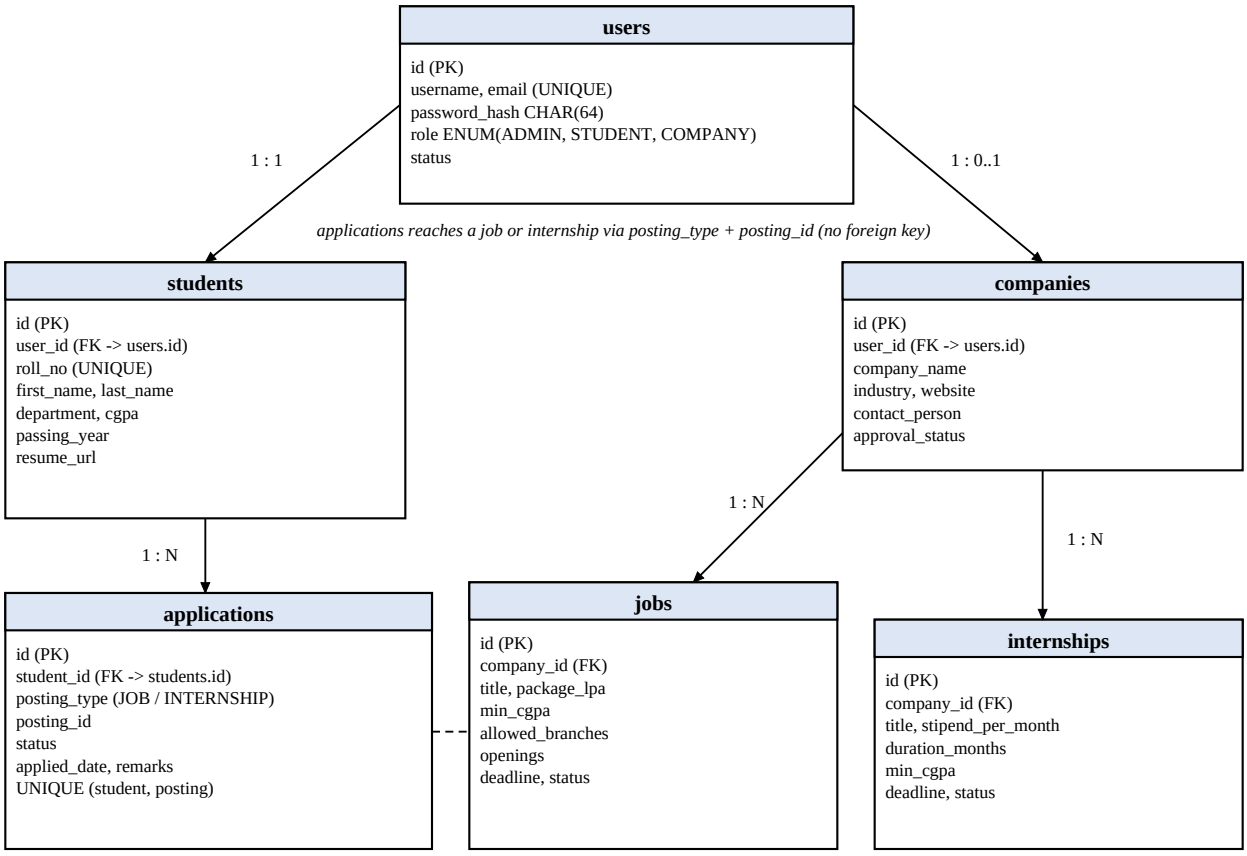


Figure 3.10: Entity–Relationship Diagram

Entity / Table	Key Fields	Constraints / Relationships
users	id (PK), username, email, password_hash CHAR(64), role (ADMIN, STUDENT, COMPANY), status	Username and e-mail unique; role constrained to the three values; parent of students and companies.
students	id (PK), user_id (FK), roll_no, first_name, last_name, department, cgpa, passing_year, resume_url	roll_no unique; one student per user; CGPA range 0.00–10.00.
companies	id (PK), user_id (FK), company_name, industry, website, contact_person, approval_status	approval_status in PENDING, APPROVED, REJECTED; owns many postings.
jobs	id (PK), company_id (FK), title, package_lpa, min_cgpa, allowed_branches, openings, deadline, status	Belongs to one company; eligibility inputs: min_cgpa, allowed_branches.
internships	id (PK), company_id (FK), title, stipend_per_month, duration_months, min_cgpa, deadline, status	Belongs to one company.
applications	id (PK), student_id (FK), posting_type (JOB / INTERNSHIP), posting_id, status, applied_date, remarks	UNIQUE (student, posting); references a posting by type and id (no FK).

- The database shall preserve referential integrity among users, students, companies, postings and applications.

- The application shall reject duplicate usernames, e-mail addresses, roll numbers and duplicate applications.
- The application shall never store passwords in plaintext.
- The student registration form collects mobile number, technical skills and address; the schema shall be updated to store them or the form fields removed [CONFIRM].
- The application shall not store résumé files; only a résumé link is stored, and third-party storage is outside its control.

3.5 Software System Attributes

3.5.1 Security

- **NFR-SEC-01:** Passwords shall never be stored in plaintext. The current prototype stores an unsalted SHA-256 digest, which is fast to brute-force; it shall be replaced by bcrypt, scrypt, Argon2id or salted PBKDF2 [2],[3].
- **NFR-SEC-02:** All queries shall use prepared statements; no user input shall be concatenated into SQL.
- **NFR-SEC-03:** AuthFilter shall protect /student/*, /company/*, /admin/*; JSP views shall reside under /WEB-INF/views/.
- **NFR-SEC-04:** Each protected record operation shall verify ownership (for example, a recruiter shall not review another company's applicants by changing an identifier).
- **NFR-SEC-05:** Session protections — session identifier renewal after login, invalidation at logout, cookie flags and idle timeout — and CSRF protection and output encoding shall be implemented and verified [CONFIRM].
- **NFR-SEC-06:** The hosted deployment shall use HTTPS and TLS to the database; secrets and credentials shall stay out of public source history; default administrator credentials shall be changed before shared deployment.
- **NFR-SEC-07:** No audit log is currently described; an audit log of status changes is a planned requirement.

3.5.2 Reliability

- **NFR-REL-01:** Invalid input, unauthorised access, missing records and database failures shall produce a controlled message, not a stack trace containing connection details.
- **NFR-REL-02:** A failed save shall not display a success confirmation, and failed operations shall have no side effects.
- **NFR-REL-03:** Status updates and related records should be written in a transaction where they must succeed together.
- **NFR-REL-04:** The duplicate-application constraint shall hold under concurrent submissions.

3.5.3 Maintainability

- **NFR-MNT-01:** The application shall retain separation between JSP presentation, servlet logic and DAO/JDBC persistence.
- **NFR-MNT-02:** Servlets, DAOs and the AuthFilter shall be independently testable.
- **NFR-MNT-03:** SQL scripts (database/placement_system.sql and database/placement_system_cloud.sql), the Dockerfile and the source shall be version-controlled in

the repository.

- **NFR-MNT-04:** Features normally supplied by frameworks (CSRF, password encoders, input binding) shall be added and tested by hand and documented.

3.5.4 Availability

- **NFR-AVL-01:** For an academic demonstration, the application should remain available during a scheduled demonstration when the host and database are operational.
- **NFR-AVL-02:** Free-tier hosting may cause cold starts and sleep; this limitation shall be documented to operators.
- **NFR-AVL-03:** The current system does not define high availability, clustering, automated disaster recovery or an uptime SLA.

3.5.5 Usability and Accessibility

- **NFR-USE-01:** Forms shall mark required fields, give field-specific error messages and remain usable on small screens.
- **NFR-USE-02:** Student-visible status labels shall be clear enough to distinguish review from an offer.
- **NFR-USE-03:** Eligibility rules shall be published, and the institution shall provide a route to appeal automated exclusions.

4. Appendices & References

Appendix A — Key Business Rules

Rule ID	Rule
BR-ELIG-01	A student is eligible for a job when CGPA ≥ min_cgpa and the branch is allowed (or "All Branches"); internships use the minimum-CGPA rule.
BR-ELIG-02	Eligibility is evaluated on the server at listing time and again at submission time.
BR-APP-01	At most one application exists for the same student and posting.
BR-APP-02	A new application starts in status PENDING.
BR-LIFE-01	Application status moves along PENDING → SHORTLISTED → INTERVIEW_SCHEDULED → ACCEPTED or REJECTED.
BR-COMP-01	A company can log in and post only after the TPO sets its approval status to APPROVED.
BR-AUTH-01	Role STUDENT may use /student/*, COMPANY /company/*, ADMIN /admin/*; any other combination is redirected to login.
BR-CGPA-01	CGPA is entered in the range 0.00–10.00 and is only as accurate as the entered value; the TPO verifies it.
BR-DATA-01	A recruiter sees only applicants to own postings; a student sees only own applications and eligible postings.

Appendix B — Edge Cases and Expected Behaviour

CampusHire shall handle invalid inputs and exceptional conditions safely. The following cases describe expected behaviour:

- **Ineligible application through a crafted request:** the server shall reject it even if the client-side check is bypassed; no record is stored.
- **Duplicate application:** the uniqueness constraint shall reject the second request and exactly one application shall remain, even under concurrent requests.
- **Posting closed or deadline passed:** the application shall be rejected according to the deadline and timezone policy.
- **Company not yet approved:** login shall be rejected until the TPO approves the company.
- **Direct URL access to another role's path:** AuthFilter shall redirect to the login page.
- **Altered posting or applicant identifier:** a recruiter shall be denied access to another company's applicants.
- **SQL injection payloads in form fields:** prepared statements shall neutralise them; no successful injection shall occur.
- **Unsupported status value or illegal transition:** the request shall be rejected rather than storing arbitrary text.
- **Database unavailable:** a controlled error shall be shown and no success message displayed.
- **Cold start on free-tier hosting:** the first request may be slow; operators shall be informed.

Appendix C — Recorded Verification Cases

The cases below were recorded by the project team in its own environments. Conditions still to be stated: number of repetitions, how the mean was computed, client location, hardware, whether the hosting plan was free tier and tool versions [CONFIRM]. They are not a load test, a user study or a security certification.

Verification Case	Recorded Result	Method / Status
Direct URL access to /student/*, /admin/* without authorisation	All unauthorised requests redirected (local and cloud)	Manual URL requests — recorded by team
Injection payloads against input fields	No successful injection observed (local and cloud)	Scripted payloads — recorded by team
Average page response, GET /	14.2 ms local; 185 ms cloud	curl / browser developer tools
Login request latency	22.6 ms local; 210 ms cloud	Request timing incl. hash and database round trip
Database query time	4.8 ms local; 42.5 ms cloud	Timing around a query call
WAR size	2.45 MB	Maven build output
Container startup	1.75 s local; 5.2 s cloud	Tomcat log
Idle JVM memory	168 MB local; 210 MB cloud	Process memory inspection
Functional test pass rate per use case	Not yet executed	Planned
Concurrent duplicate submissions; boundary CGPA values	Not yet executed	Planned
Illegal status transitions	Not yet executed	Planned
Independent OWASP Top 10 scan; usability test (SUS)	Not yet executed	Planned

Appendix D — Requirement Traceability Overview

Feature Area	Requirement IDs	Verification Evidence
Authentication and roles	FR-AUTH-01 to FR-AUTH-06; NFR-SEC-*	Registration/login tests; invalid-credential test; role-by-path request matrix; direct URL tests.
Student profile and postings	FR-STU-01 to FR-STU-03; FR-CMP-01 to FR-CMP-04	Profile CRUD tests; company approval test; ownership tests.
Eligibility and applications	FR-ELG-01, FR-ELG-02; FR-APP-01 to FR-APP-03	CGPA boundary tests; crafted-request test; concurrent duplicate test.
Application lifecycle	FR-CMP-05; FR-LIF-01 to FR-LIF-03	Walk every legal and illegal transition; student status view check.
Administration and UI	FR-ADM-01 to FR-ADM-03; FR-UI-01; FR-ENV-01	Approval flow test; statistics definition check; local vs cloud database switch.
Performance and quality	NFR-PERF-*; NFR-REL-*; NFR-MNT-*; NFR-AVL-*	Latency sampling, load test, negative tests and deployment observation.

Appendix E — Deployment and Operational Notes

- Deployed prototype: https://campushire-fdo8.onrender.com; source: https://github.com/Uditbhardwajj/JAVA_project [CONFIRM: repository link].

- Build with Maven into one WAR; a multi-stage Docker build packages it into an Apache Tomcat 10.1 image, hosted on Render with a TiDB Cloud Serverless database.
- Use database/placement_system.sql for local MySQL and database/placement_system_cloud.sql for the cloud database; DBConnection selects the setting from configuration.
- Keep credentials outside public source history and change default administrator credentials before shared deployment.
- Free-tier hosting may sleep; expect cold-start delay on the first request.
- Before real use: replace password hashing, add CSRF tokens and secure cookie flags, verify output encoding, add an audit log, and review data-protection obligations with qualified persons.

5. References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- OWASP Foundation, “OWASP Top 10:2021 — The ten most critical web application security risks”: https://owasp.org/Top10/
- P. A. Grassi et al., “Digital identity guidelines: Authentication and lifecycle management,” NIST Special Publication 800-63B, 2017 (revised 2020).
- OWASP Foundation, “Password Storage Cheat Sheet”: https://cheatsheetseries.owasp.org/
- OWASP Foundation, “SQL Injection Prevention Cheat Sheet”: https://cheatsheetseries.owasp.org/
- Eclipse Foundation, “Jakarta Servlet Specification, Version 6.0,” 2022: https://jakarta.ee/specifications/servlet/6.0/
- Apache Software Foundation, Apache Tomcat Application Developer’s Guide: https://tomcat.apache.org/tomcat-10.1-doc/appdev/index.html
- Oracle, The Java Tutorials: Using Prepared Statements (JDBC): https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html
- OWASP Foundation, “Session Management Cheat Sheet” and “Authorization Cheat Sheet”: https://cheatsheetseries.owasp.org/
- R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.
- National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.
- MySQL Reference Manual: https://dev.mysql.com/doc/
- CampusHire project report and research paper supplied with this request: architecture, algorithms, schema, measurements and limitations.

Approval / Review

Role	Name	Date
Prepared by Team	Virendra Singh Chundawat, Vaibhav Charan, Yash Kumar, Udit Bharadwaj, Rahul Tailor	
Reviewed by Faculty	Er. Ram Babu Buri	
Head of Department	Dr. Ashok Kajla	
Approved for Submission		